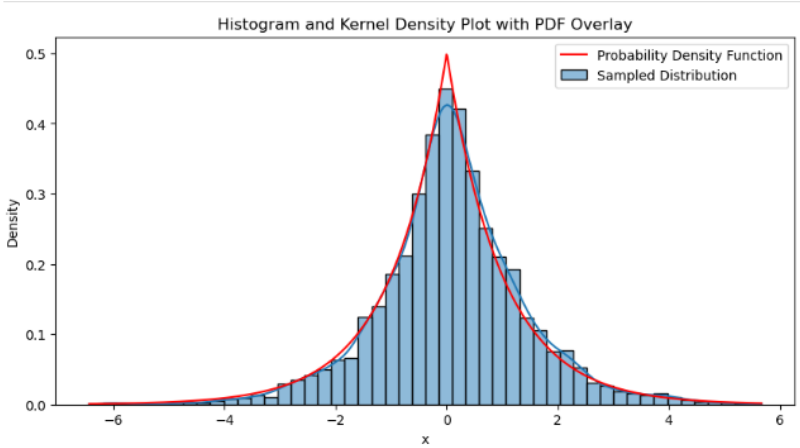
ST2195 PROGRAMMING REPORT

DIVYA NISTALA
220658043
ST2195- Programming for Data Science

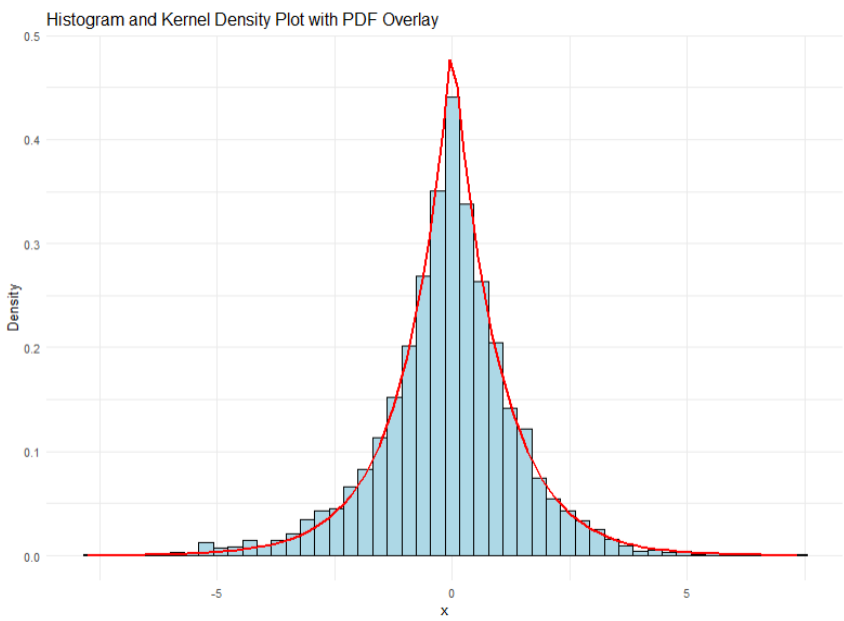
Part 1 (a)

The Metropolis-Hastings algorithm is a sampling technique from the Markov Chain Monte Carlo algorithm. It is used to generate random numbers for the distribution with a probability density function (PDF) of $f = \frac{1}{2}\exp(-|x|)$. The random walk Metropolis algorithm runs N=10 000 chains following a Normal distribution with standard deviation (s), and the simulated results are illustrated using a Histogram and Kernel Density plot. The overlay of the PDF along the plots clearly visualises the agreement with the estimates, showing the trueness of the values.




```
Sample Mean: 0.07914833001526135
Sample Standard Deviation: 1.315989281687335
```

This is the plot created using Python. The sample mean generated is 0.0791483 and the sample standard deviation generated is 1.315989.

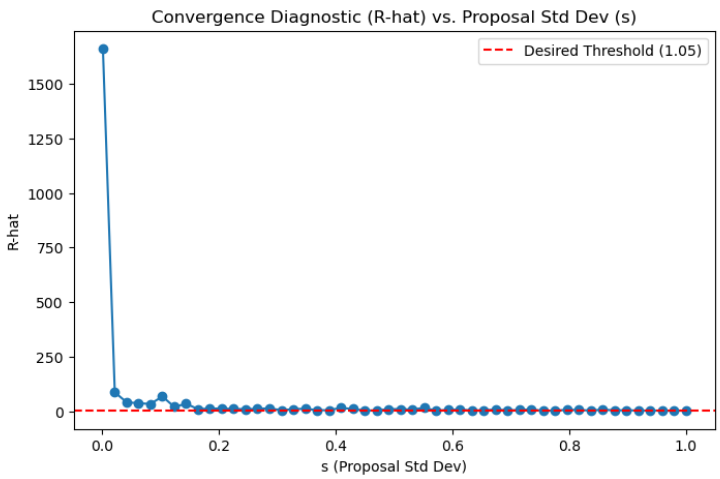


This is the plot created using R. The sample mean generated is -0.0943715 and the sample standard deviation generated is 1.481383.

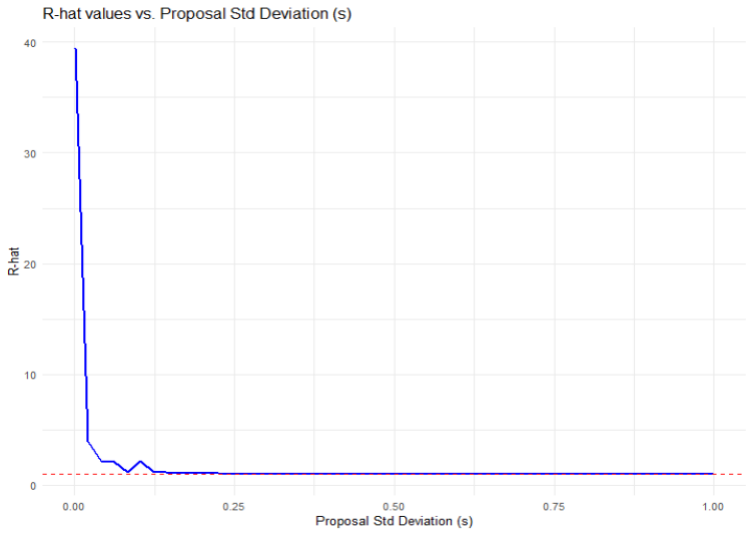
Part 1 (b)

The Rhat diagnostic is used in this analysis to measure the convergence of the variance inside and between various chains in the Metropolis-Hastings algorithm. We generate four unique Markov chains with differing starting values and with different s values ranging from 0.001 to 1. This estimation is generated using N= 2000, with 2000 varying iterations. For each chain generated, the Rhat value is determined by calculating the sample mean (Mj) and variance (Vj), followed by the overall mean (M), within-chain variance (W), between-chain variance (B), and the Rb value. The visualization of the convergence behavior of the chains (Rhat) versus the s values is illustrated through a plot, as shown in the figure below.

This analysis is useful in determining the right s values for the Metropolis-Hastings algorithm to generate more accurate and reliable simulations.



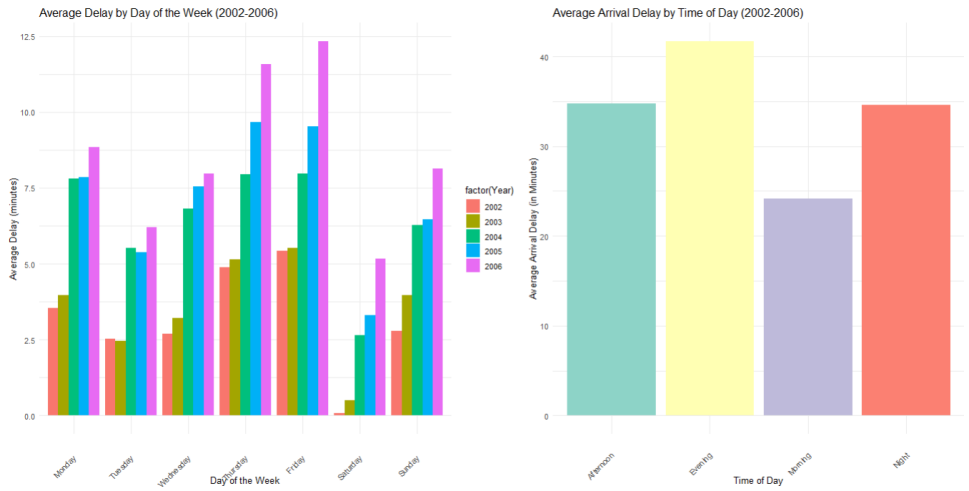
This is the plot created using Python. The plot clearly illustrates that for a small s value, the Rhat value is high, showing poor convergence behavior. For larger s values, the Rhat value is closer to 1, showing better convergence of the chains.



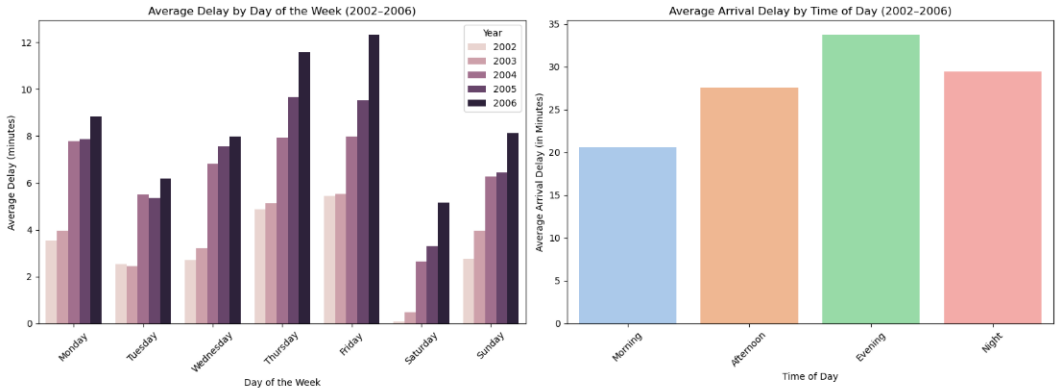
This is the plot created using R. The plot clearly illustrates that for a small s value, the Rhat value is high, showing poor convergence behavior. For larger s values, the Rhat value is closer to 1, showing better convergence of the chains.

Part 2 (a)

The 5 consecutive years selected to analyse are 2002 to 2006. We start off by importing the necessary packages into R and Python. The working directory to the Harvard Dataverse files is then set up, and an SQLite database is created to load the flight data yearly in yearly.csv.bz2 files.



2a R Plot



2a Python Plot

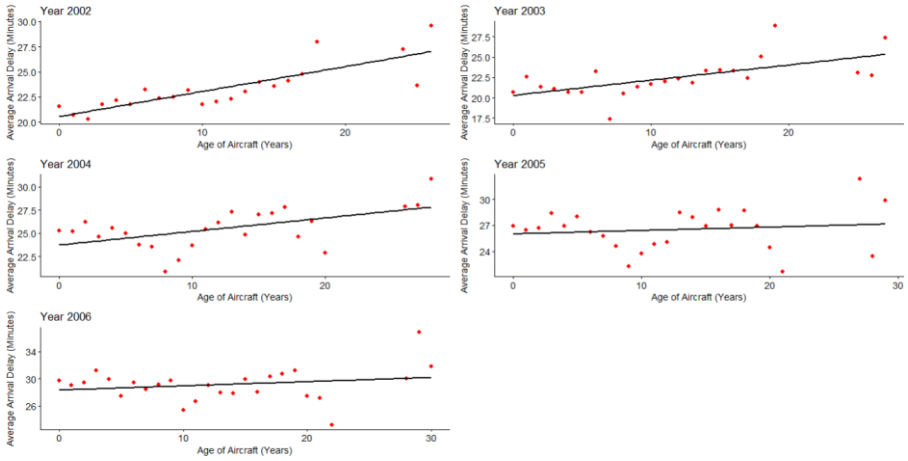
This question asks us to determine the best times and days of the week to minimise flight delays. We then use the functions 'Year', 'DayOfWeek', 'CRSDepTime' and 'ArrDelay' to filter out and extract the necessary information from the data to be analysed for the question. First, we use 'Year' and 'DayOfWeek' to compare average delay times yearly against the days of the week. Then, we use 'CRSDepTime' and 'ArrDelay' to segment each day into 4 timings: Morning, Afternoon, Evening and Night, and compare these timings against average arrival delay times. To plot the graphs, we use ggplot2 for R and seaborn for Python.

As seen from the plot comparisons from both R and Python plots above, we can conclude that the best days to fly are Tuesdays to Thursdays, and the best timings are in the mornings.

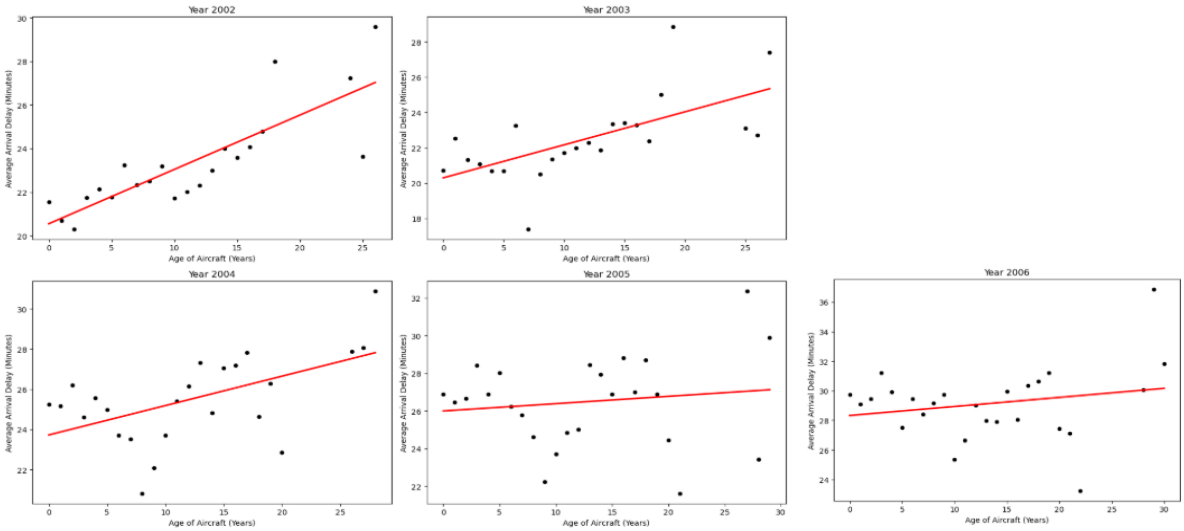
Part 2 (b)

This question asks us to determine if older planes face more delays. We use the functions 'issue_date' and 'issue_year' to filter the plane-data.csv file into proper dates of issue and manufacturing of the planes. We further filter it with 'TailNum', and 'ArrDelay' to ensure rows of data with missing dates, inaccurate tail numbers and negative arrival values were excluded. This filtered data is then added back to the plane database to ensure the data is most accurate to calculate the plane age by subtracting its manufacturing year from its issue year.

The plane age is merged with average delay using 'plane_age' to plot average delay time against aircraft age in a scatterplot with a regression line to show the correlation trend for each year. This can be seen from the plots below from R and Python.



2b R Plot



2b Python Plot

The Pearson Correlation Coefficient is calculated, as seen below, to generate a value for the relationship for each year. These coefficient values are classified as either strong, moderate or weak to describe the relationship.

```
[1] "Year 2002 : A correlation coefficient of 0.83 indicates a moderate positive correlation between aircraft age
and arrival delay."
[1] "Year 2003 : A correlation coefficient of 0.63 indicates a moderate positive correlation between aircraft age
and arrival delay."
[1] "Year 2004 : A correlation coefficient of 0.55 indicates a moderate positive correlation between aircraft age
and arrival delay."
[1] "Year 2005 : A correlation coefficient of 0.13 indicates a weak positive correlation between aircraft age and
arrival delay."
[1] "Year 2006 : A correlation coefficient of 0.22 indicates a weak positive correlation between aircraft age and
arrival delay."
```

2b R Plot Correlation Conclusion

```
Processing year: 2002
Year 2002: A correlation coefficient of 0.83 indicates a moderate positive correlation between aircraft age and arrival delay.
Processing year: 2003
Year 2003: A correlation coefficient of 0.63 indicates a moderate positive correlation between aircraft age and arrival delay.
Processing year: 2004
Year 2004: A correlation coefficient of 0.55 indicates a moderate positive correlation between aircraft age and arrival delay.
Processing year: 2005
Year 2005: A correlation coefficient of 0.13 indicates a weak positive correlation between aircraft age and arrival delay.
Processing year: 2006
Year 2006: A correlation coefficient of 0.22 indicates a weak positive correlation between aircraft age and arrival delay.
```

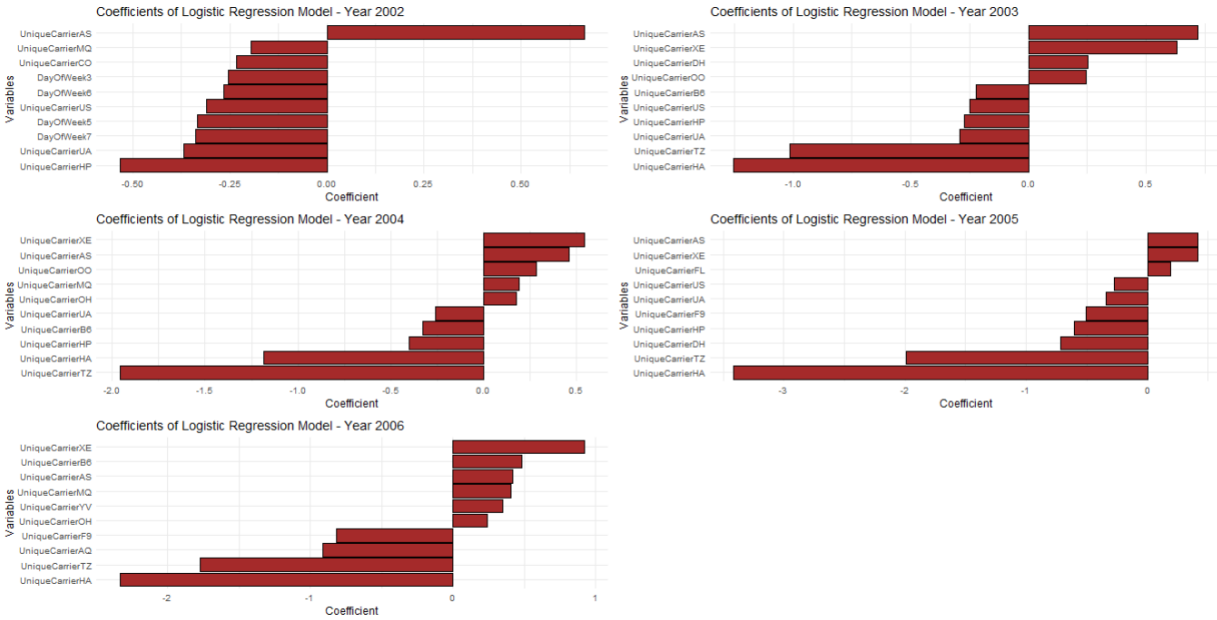
2b Python Correlation Conclusion

These outputs show consistent moderate positive correlations for most years, and some weak positive correlations for some years. The moderate values suggest that older planes do tend to experience more delays, while the weaker values suggest that delays may be less due to plane age. Hence, we cannot definitively conclude that older planes experience more delays, nevertheless, it could be a large contributing factor.

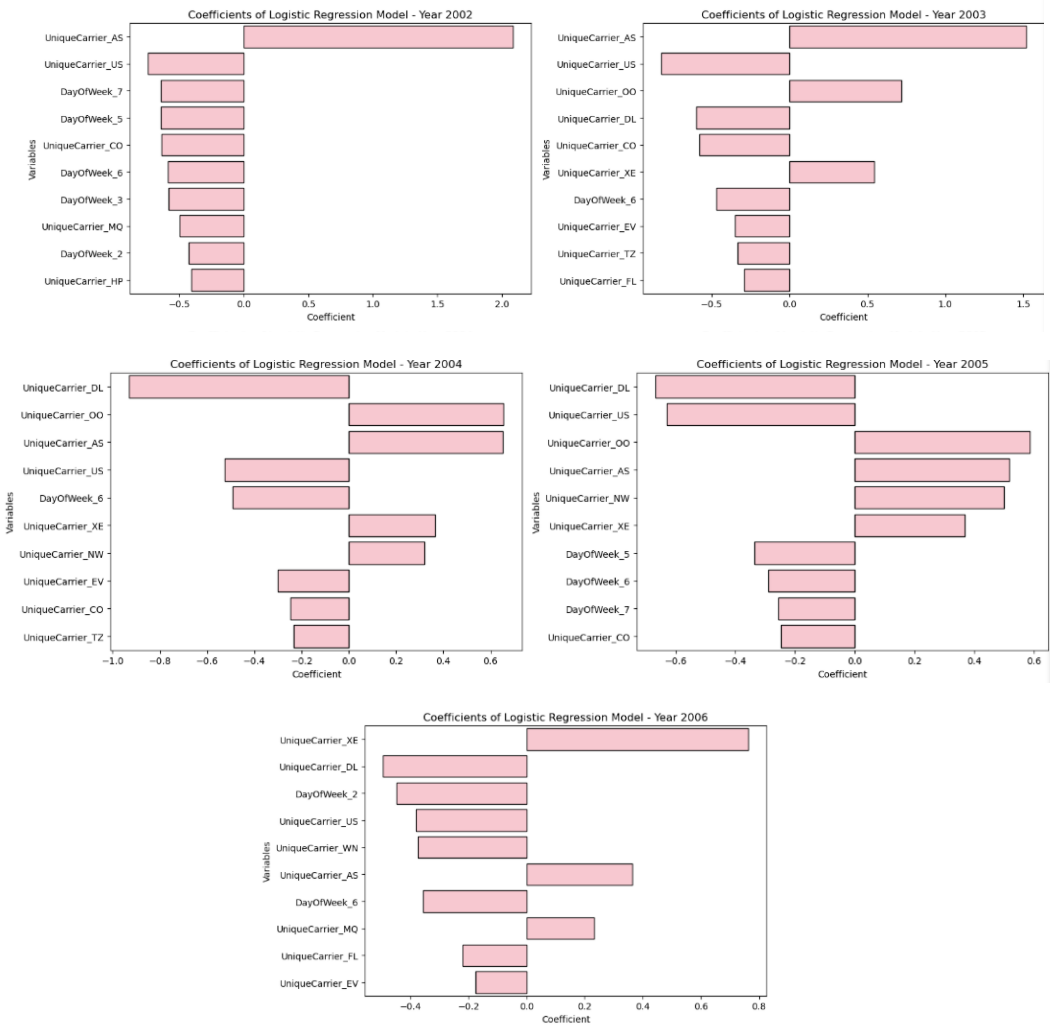
<u>Part 2 (c)</u>

This question asks us to fit a logistic regression model for the probability of US diverted flights, using as much information from the dataverse as possible. To prepare the data, we use the functions ‘Diverted’, ‘Month’, ‘DayOfWeek’, ‘CRSDepTime’, ‘CRSArrTime’, ‘Distance’, ‘Origin’, ‘Dest’, and ‘UniqueCarrier’ to filter out this data as predictors from each year’s .csv.bz2 files and make sure null values are excluded. We then use ‘dep_hour’ and ‘arr_hour’ to convert timings into hours for easier application and combine the cleaned and filtered data it with the airports_csv dataset. Lastly, we use ‘diverted_flag’ to spotlight diverted flights, making it more fitting for the plotting analysis.

We use the function ‘glm()’ in R to produce bar plots with the most popular 10 predictors plotted against their respective obtained coefficient values. In Python, we first use Pipeline to clean numerical data and also to convert variables such as ‘UniqueCarrier’ into numbers to ensure accuracy, and then use 'lbfgs' to plot a clear Linear Regression model. The variable ‘UniqueCarrier’ is set up to contain each separate predictor to allow the model to run smoother in each language. The plot outputs for R and Python can be seen from the figures below.



2c R



2c Python

These plots illustrate the extent of influence on flight diversion by the direction of the plot. If the influence on probable flight diversion is high, the bar plots against positive coefficients to the right side, and if the influence on probable flight diversion is low, the bar plots against negative coefficients to the left side.

Both the R and Python plots provide a clear visualisation of how different variables can influence airport carriers flight diversion times. Hence, these logistic regression models provide a better understanding of flight performances influenced by operational factors.

Conclusion

In this analysis for the years 2002 to 2006, we have obtained the best times and days of the week to minimise flight delays, investigated whether the plane age affects delay probability, and have applied modelling to further evaluate how operational variables influence the probability of flight delays. In conclusion, we now have a better understanding of flight performance.